

SIR SYED UNIVERSITY OF ENGINEERING & TECHNOLOGY
COMPUTER SCIENCE & INFORMATION TECHNOLOGY DEPARTMENT

Fall 2023

Data Structures and Algorithms (CS-212T) Assignment # 1

Name: Ahmed Mallick

Roll no.: 2022F-BCS-154

Section: C

Total Marks: 4

Due Date: 08-11-2023

Instructor Name: Adeel Saeed

CLO #	Course Learning Outcomes (CLOs)	PLO Mapping	Bloom's Taxonomy
CLO 1	Understand fundamental data structures as well as algorithms and their complexities	PLO_1 (Knowledge)	C2 (Understanding)

Q1. What is the running time of the following algorithm (using Big-O notation). Show your analysis in a detailed analysis which counts the number of times each line of code executes.

ANSWER:

```
1-    int sum = 0;                // Executes once for (i = 0; i <
n; i++) {                        // Executes n times for (j = 0; j < n; j++) {
// Executes n^2 times for (k = 0; k < n; k++) {    // Executes
n^3 times if (i == j == k) {        // Executes 1 time (This
condition is usually always false) for (l = 0; l < n*n*n; l++) {
// Executes n^3 times    sum = i + j + k + l;    // Executes
n^3 * n^3 = n^6 time
}}}}
```

The total number of times the innermost assignment is executed is n^6 . So, the running time of this algorithm is $O(n^6)$ in terms of Big-O notation. However, note that the condition $\text{if } (i == j == k)$ is a constant time check, so it doesn't change the overall complexity, and this check is

usually always false in practice. If we assume this check is always false, the running time simplifies to **$O(n^6)$** .

```
2-    int array_sum(int a, int n) {    int i;    int sum = 0;    for (i =
0; i < n; i++) {        sum = sum + a[i]
    }
    return sum;
}
```

ANSWER:

The sum of elements in an integer array 'a' of size 'n'. To analyze its running time using Big-O notation, we'll count the number of times each line of code executes. The initialization of 'i' and 'sum' each takes constant time ($O(1)$). The for loop iterates 'n' times, and within the loop, the addition operation ('sum = sum + a[i]') is performed 'n' times. So, the dominant factor in the time complexity is the loop, which executes $O(n)$ times. Therefore, the overall time complexity of this algorithm is **$O(n)$** , indicating that the running time grows linearly with the size of the input array 'n'.

Q2. Show the series of steps taken by the Bubble Sort and Selection Sort algorithm while sorting following list in ascending order.

index	0	1	2	3	4	5	6	7
value	9	20	6	10	14	8	60	11

ANSWER:

Bubble Sort algorithm to sort the given list in ascending order.

Original List:

Index: 0 1 2 3 4 5 6 7

Value: 9 20 6 10 14 8 60 11

Step 1:

Compare elements at index 0 and 1 (9 and 20). Swap them because 9 is greater.

Index: 0 1 2 3 4 5 6 7

Value: 9 20 6 10 14 8 60 11

Step 2:

Compare elements at index 1 and 2 (20 and 6). Swap them because 20 is greater. Index:

0 1 2 3 4 5 6 7

Value: 9 6 20 10 14 8 60 11

Step 3:

Compare elements at index 2 and 3 (20 and 10). Swap them because 20 is greater.

Index: 0 1 2 3 4 5 6 7

Value: 9 6 10 20 14 8 60 11

Step 4:

Continue comparing and swapping elements until you reach the end of the list.

Index: 0 1 2 3 4 5 6 7

Value: 9 6 10 14 20 8 60 11

Step 5:

Repeat the process for the entire list again. Continue until no more swaps are needed. Index:

0 1 2 3 4 5 6 7

Value: 6 9 10 14 8 20 60 11

Index: 0 1 2 3 4 5 6 7

Value: 6 9 10 8 14 20 60 11

Index: 0 1 2 3 4 5 6 7

Value: 6 9 10 8 14 20 60 11

Index: 0 1 2 3 4 5 6 7

Value: 6 9 8 10 14 20 60 11

Index: 0 1 2 3 4 5 6 7

Value: 6 9 8 10 14 20 11 60

Index: 0 1 2 3 4 5 6 7

Value: 6 9 8 10 14 11 20 60

Step 6:

The list is now sorted. No more swaps are needed.

Index: 0 1 2 3 4 5 6 7

Value: 6 8 9 10 11 14 20 60 The Bubble Sort algorithm has successfully sorted the list in ascending order.

Selection Sort algorithm as it sorts the given list in ascending order.

Step 1:

Start with the original list. [9,
20, 6, 10, 14, 8, 60, 11]

Step 2:

Find the minimum element in the unsorted part of the list and swap it with the first element.
[6, 20, 9, 10, 14, 8, 60, 11]

Step 3:

Move to the next unsorted element (index 1) and repeat the process of finding the minimum element and swapping it with the element at the current index.
[6, 8, 9, 10, 14, 20, 60, 11]

Step 4:

Continue this process for each remaining unsorted element in the list, incrementing the current index each time.
[6, 8, 9, 10, 11, 14, 60, 20]

Step 5:

The list is now sorted in ascending order.

So, the sorted list using the Selection Sort algorithm is:

[6, 8, 9, 10, 11, 14, 20, 60]